

DQ Compass

The Audit Layer — what we implemented

MBA-ESG / SG GSC Datathon 2026 — *DQ Compass: a Plug-and-Play Data Quality Control Layer for EUCs*

Working Group 1

Generated 2026-09-08

The Audit Layer is the brief's fourth stage — **DEFINE -> EXECUTE -> REPORT -> EVIDENCE** (section 4.4, Appendix B). Its job: make every run **traceable**, **reproducible** and **tamper-evident**, and record who reviewed it. This document lists each mechanism and the exact artefact it produces.

Mechanism	Produced by	Artefact
Evidence Pack (per run)	dq_engine.py :: DQEngine.run()	evidence// (13+ files)
Hash-chained run ledger	dq_engine.py :: _append_ledger()	evidence/runs_ledger.jsonl
Independent verification	dq_engine.py :: verify_run()	evidence//verification.json
Human sign-off	app.py :: run_signoff()	evidence//signoff.json
Catalogue change log	rule_authoring/store.py :: _log()	catalogue_changelog.jsonl
Data-source change log	rule_authoring/source_store.py :: _log()	data_sources_changelog.jsonl
Audit surfaces (UI)	app.py routes /runs, /runs/, /activity	browsable + downloadable

1 The Evidence Pack — evidence/<run_id>/

Every execution of the engine creates one folder, named by a random UUID4 run_id. Nothing is overwritten: a new run is a new folder. The folder covers the ten components Appendix B.2 asks for.

#	Appendix B.2 component	File(s)	Content
1	Run identification	run_summary.json	run_id (UUID4), run_timestamp_utc (ISO-8601 UTC), engine_version, catalogue_source; every per-rule record repeats execution_timestamp_utc
2	Dataset reference	run_summary.json, _result.json	dataset_reference = {name: content hash}. Hash = SHA-256 of pandas.util.hash_pandas_object(df, index=True), first 16 hex - a fingerprint of the exact data evaluated
3	Dataset snapshot	snapshots/.csv	an exact copy of every input dataframe actually loaded, written once per dataset (df.to_csv, index=False)
4	Rule configuration snapshot	_result.json .rule_configuration_snapshot	dataset_scope, ref_dataset_scope, params (as executed), threshold_pct - the rule exactly as it was at run time
5	Execution parameters (resolved)	_result.json .execution_parameters	the effective values: e.g. reference_date 'today' -> reference_date_resolved '2026-09-08'; tolerance_pct and threshold_pct surfaced
6	Control results	_result.json	status (PASS / FAIL / ERROR), metrics {}, control_function name, exception_count, severity, owner
7	Exception dataset	_exceptions.csv	every failing row, written only when there is at least one; the path is recorded as exceptions_evidence_path
8	Processing log + system trace	run_log.jsonl, run.log, system_trace.json	ordered events (structured + text mirror) and the environment fingerprint (see 1.1)
9	Run integrity	run_summary.json .previous_run_hash / .run_hash	the link into the hash chain - see section 2
10	Sign-off	signoff.json	reviewed_by, reviewed_at, decision, comment - written from the Runs page (section 4)

A verification.json also appears once a verification has been run (section 3).

1.1 system_trace.json - the environment fingerprint

So a reviewer can tell whether a difference comes from the data or from the tools:

- **engine_version, python_version, platform, host**
- **library versions:** pandas, pyyaml, openpyxl
- **SHA-256 (16 hex) of** control_catalogue.csv, datasets_config.yaml and controls.py - pin the exact rule set and the exact control code used
- **controls_used:** the control function name mapped to each rule id
- **retention_note:** states that a production deployment would store these files on WORM storage (S3 Object Lock / immutable data lake)

1.2 run_log.jsonl / run.log - the processing trace

One line per step, in order, both machine-readable (JSONL) and as plain text. Events: run_start, dataset_loaded (rows, columns, hash, snapshot path), rule_start, rule_done (status, exception count) or rule_error, rule_skipped (inactive rule), system_trace_written, run_complete (run_hash).

1.3 ERROR records - the run never crashes

If a rule's control type is unknown, or its data source is missing / retired, the engine writes a status: "ERROR" record with the reason and **carries on with the next rule** (_error_record). The evidence is still complete; the reporting scores that rule as a total miss.

2 The hash-chained run ledger — evidence/runs_ledger.jsonl

An append-only file, one JSON line per run, in run order. Each line:

```
{ "run_id", "timestamp", "engine_version", "catalogue_version_hash",  
  "rules_executed", "counts": {PASS, FAIL, ERROR},  
  "previous_run_hash", "run_hash" }
```

2.1 How run hash is computed

```
payload = canonical_json( run_summary without the previous_run_hash key )  
# canonical_json = json.dumps(..., sort_keys=True) -> order-independent  
  
run_hash = SHA-256( previous_run_hash + payload ).hexdigest()[:16]  
  
previous_run_hash = run_hash of the last line already in the ledger  
( "" for the very first run )
```

So run_hash depends on **the previous run's hash** plus **the entire content of this run's summary** - every rule status, every metric, every config snapshot, every dataset content hash.

2.2 What it guarantees

- **Tamper-evident:** change one byte in any past run_summary.json, or delete / reorder / insert a run, and that run's recomputed run_hash no longer matches the ledger, and every later line's previous_run_hash no longer matches either - the break is visible and localised.
- **Append-only history:** the next run always chains onto the last ledger line, so the sequence of runs is fixed once written.
- **Config drift is visible:** catalogue_version_hash on each line is the SHA-256 of the catalogue at that run - you can see the rule set changing between runs.

Prototype scope: hashes are truncated to 16 hex (64-bit) for readability, and the chain detects tampering but is not itself cryptographically signed. Production would use full 256-bit digests and an HMAC / signature per line.

3 Independent verification — verify_run() / Appendix B.3

Proves that a recorded run is **reproducible from its own evidence alone** - not from the live catalogue or the live data.

3.1 What it does

For the chosen run it reads `run_summary.json` and, for every non-ERROR rule result:

- looks up the control function by the recorded `control_type`
- reads the stored snapshots/`<dataset>.csv` (via `_read_snapshot`, which uses `engine="python"` so it parses exactly as the engine's CSV connector did - a mixed-type column must not re-infer a different dtype)
- re-executes the function with the **stored** params (and the stored reference snapshot if the rule has one)
- re-applies the **stored** `threshold_pct`
- compares the recomputed status and `exception_count` against what was recorded

ERROR records are reported as “not re-executed” and count as a match (there is nothing to recompute).

3.2 Output - verification.json

```
{ "run_id", "verified_at", "run_hash",  
  "method": "re-execute stored rule config against stored dataset snapshots",  
  "all_match": true | false,  
  "checks": [ { rule_id, expected, recomputed,  
    expected_exceptions, recomputed_exceptions, match } , ... ] }
```

3.3 How to run it

Surface	Command / action
CLI (engine)	<code>python dq_engine.py --verify</code> (exit 0 if reproducible, 1 if not)
CLI (packaged)	<code>dqcompass verify</code>
Web app	"Verify this run" button on <code>/runs/</code> - writes <code>verification.json</code> and shows the per-rule checks table
CI gate	<code>dqcompass gate --severity High</code> (fails a pipeline when a High-severity control is failing)

4 Human sign-off — signoff.json

A run is not “done” until a person accepts it. The Runs page has a small form (decision = acknowledged / accepted / rejected - remediation required, plus an optional comment). Submitting it writes:

```
{ "reviewed_by", "reviewed_at", "decision", "comment" }
```

- **reviewed_by** = the name in the top-bar “editor” field (the same identity stamped on every catalogue / source change).
- **reviewed_at** = server timestamp (ISO-8601 UTC).
- The run list then shows a **signed-off** badge; the run detail shows the decision and comment.

Prototype scope: one free-text decision, no multi-party approval workflow or RBAC.

5 Change logs — who changed what, when

The audit trail is not only about runs. Every change to a **rule** or a **data source** is journalled in an append-only JSONL file, and each entry records the **SHA-256 of the whole file before and after** the change.

Log	Written by	Actions logged	Per-entry fields
catalogue_changelog.jsonl	rule_authoring/store.py :: _log()	create, update, delete, activate, deactivate, retarget (source rename cascade)	timestamp, user, action, rule_id, note, catalogue_hash_before, catalogue_hash_after, before (rule snapshot), after (rule snapshot)
data_sources_changelog.jsonl	rule_authoring/source_store.p y :: _log()	create, update, retire, reactivate, purge, rename	timestamp, user, action, source, note, hash_before, hash_after, (+ affected rule ids on a cascade)

- **user** = the name typed in the top-bar “editor” field, stamped on every write (defaults to “anonymous”).
- Because before/after hashes are recorded, the state of the catalogue or the source registry at any past moment is reconstructible, and any edit made outside the app shows up as a file hash that no log entry explains.
- **before / after** rule snapshots make each change fully reversible by inspection.

6 Audit surfaces in the web app

Route	What it shows
/runs	Every run from the ledger: run_id, timestamp, PASS/FAIL/ERROR counts, engine version, run_hash, a verified / mismatch badge (from verification.json) and a signed-off badge. Search + filter by result / verification / sign-off, and sort.
/runs/	The full Evidence Pack: the integrity block (run_hash, previous_run_hash, catalogue hash, datasets_config hash, dataset snapshots); per-rule results that expand to metrics, config snapshot, resolved execution parameters and the exception rows (tested column highlighted, plus a How-to-fix note); the Verify button + checks table; the Sign-off form / record; the system trace; the processing-log tail. Every evidence file is downloadable.
/activity	One reverse-chronological timeline merging catalogue_changelog.jsonl and data_sources_changelog.jsonl (filter: rules / sources).
/mapping	Appendix C supervisory mapping: per rule, the requirement -> control -> evidence generated -> output, plus its last run id / status / run_hash. Ties each control to the evidence this layer produces.

7 Appendix B compliance

Requirement (Appendix B / section 4.4)	Where satisfied	Met
B.2-1 Run identification (id + timestamps)	run_summary.json, every *_result.json	YES
B.2-2 Dataset reference (content hash)	_dataset_hash -> dataset_reference	YES
B.2-3 Dataset snapshot	snapshots/.csv	YES
B.2-4 Rule configuration snapshot	*_result.json .rule_configuration_snapshot	YES
B.2-5 Execution parameters (resolved)	_resolve_execution_parameters	YES
B.2-6 Control results (status + metrics)	*_result.json	YES
B.2-7 Exception dataset	_exceptions.csv	YES
B.2-8 Processing log + system trace	run_log.jsonl, run.log, system_trace.json	YES
B.2-9 Integrity / chaining	run_hash = SHA-256(prev + canonical summary)	YES
B.2-10 Sign-off	signoff.json (Runs page)	YES
B.3 Independent verification	verify_run() -> verification.json ; dqcompass verify	YES
Tamper-evidence over the run history	append-only hash-chained runs_ledger.jsonl	YES
Change accountability (rules + sources)	two hash-chained JSONL change logs + user stamp	YES

8 Worked example (current demo, 3 runs)

The ledger for the seeded demo (14 rules on finance_data + reference sets):

```
run_id previous_run_hash run_hash counts catalogue_hash
f7647b20 (empty / 1st run) 9ff826b484d05bb7 8 PASS / 6 FAIL 68887c88a7fde846
57ba0181 9ff826b484d05bb7 d16c1fac80b843cc 8 PASS / 4 FAIL 2a5c1de63d46cbe3
0f13237e d16c1fac80b843cc b1c8ba33ffd68f60 8 PASS / 6 FAIL 256fdedd7d0b645d
```

- Each previous_run_hash equals the prior line's run_hash - the chain is intact.
- The catalogue_hash changes across the three runs because two rules were toggled off then on again between them - visible config drift.
- Latest run 0f13237e Evidence Pack: run_summary.json, system_trace.json (engine 1.0, Python 3.12.8, pandas 2.2.2), 6 dataset snapshots, 14 *_result.json, 8 *_exceptions.csv, run_log.jsonl + run.log.
- verification.json: all_match = true, 14 / 14 checks - status and exception count reproduce exactly from the stored config and snapshots.